1차 · 팀 기획서

로그 워치독 Log Watchdog
받은 데이터로 3일 안에 만드는 것

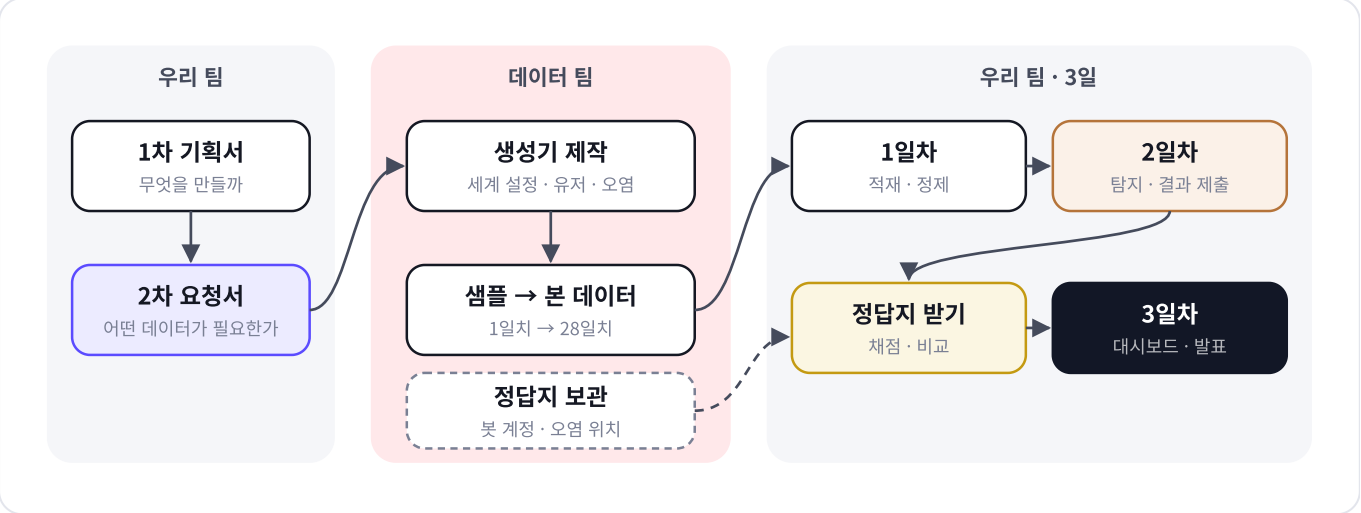
다른 팀이 만들어 줄 가상 MMORPG 로그를 받아서 **정제 → 탐지 → 시각화**까지 끝내는 3명 · 3일 프로젝트 기획서입니다. 데이터 팀에 보낼 요청서는 2차 · 데이터 요청서에 따로 있습니다.

기획 v0.2 · 2026.10 · 3인 · 작업 기간 3일 (데이터 수령 후)

---

0 프로젝트 진행 방식

우리 팀은 데이터를 직접 만들지 않습니다. 요청서를 보내면 다른 팀이 오염이 섞인 가상 로그를 만들어 주고, 우리는 그 데이터를 받아 3일 안에 결과물을 완성합니다. 데이터 팀은 정답지(어떤 계정이 봇인지, 어디에 오염을 넣었는지)를 따로 보관하고, 우리 결과를 채점하는 데 씁니다.



1 무엇을 만드는가

MMORPG "◯◯온라인"의 유저 로그를 정제하고, 매크로 봇 · 작업장 · 현금 거래(RMT) 계정을 찾아내는 데이터 파이프라인과 운영 대시보드입니다.

정제 없이는 탐지도 없다

중복 이벤트 하나 때문에 정상 유저가 "초당 사냥 10회"로 집계돼 봇으로 오탐됩니다. 엔씨소프트도 대회 데이터를 낼 때마다 오류가 발견돼 다시 추출했습니다(별첨 B-11). 이 프로젝트는 정제 전후로

탐지 결과가 어떻게 달라지는지를 숫자로 보여 줍니다.

최종 산출물 5가지

01

메달리온 파이프라인

받은 파일을 Bronze → Silver → Gold로 적재·정제하는 코드. 명령 한 줄로 처음부터 다시 돌릴 수 있어야 합니다.

02

데이터 품질 리포트

오염 유형별로 몇 건을 찾았고 어떻게 처리했는지 정리한 리포트. 데이터 팀 정답지와 비교한 탐지율도 포함합니다.

03

봇 · 작업장 탐지 결과

계정별 위험 점수와 탐지 사유. 정답지 대비 정밀도와 재현율, 정제 전후 오탐률 비교.

04

운영 대시보드

Streamlit 웹 대시보드 5개 화면: 현황, 데이터 품질, 의심 계정, 계정 타임라인, 거래 네트워크.

05

발표 자료 · README

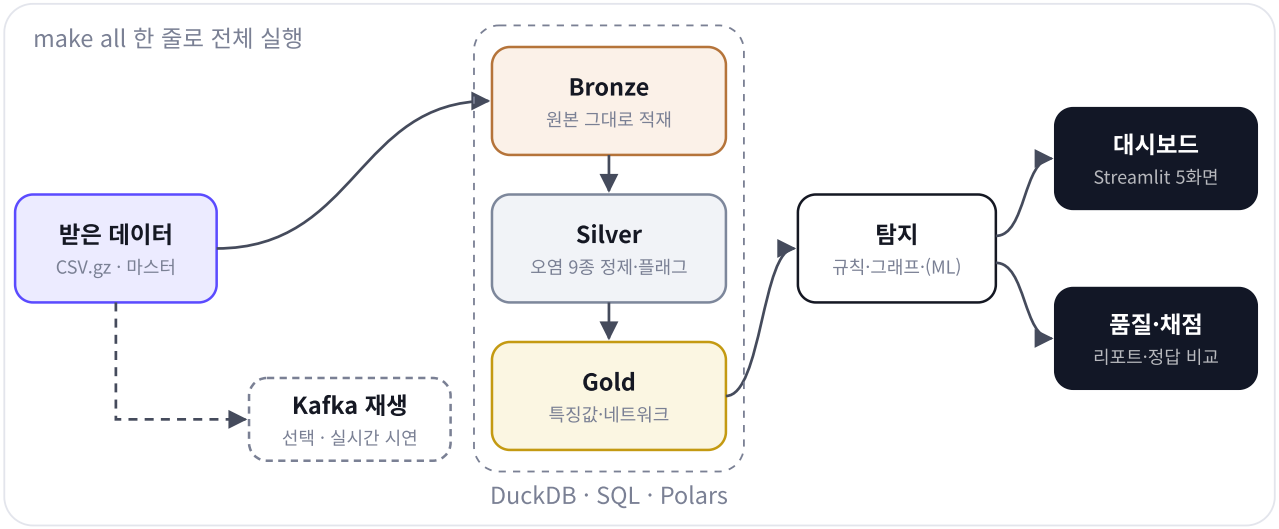
문제 정의, 구조, 결과 수치, 시연. GitHub 저장소 README에 실행 방법과 데이터 출처를 적습니다.

완료 기준

3일차 저녁까지

새 PC에서 README대로 실행하면 대시보드까지 뜨는 상태. 시연 영상 백업 포함.

2 구조



Bronze · 원본 보존

- 받은 Parquet/CSV를 그대로 DuckDB에 적재
- 파일명, 적재 시각, 원본 행 번호 추가
- 절대 수정하지 않음 (재처리 기준점)

Silver · 정제

- 9가지 오염 처리 (아래 표)
- 세션 재구성, 타입 · 시간대 통일
- 행을 지우지 않고 품질 플래그 컬럼으로 표시
- 형식이 깨진 행만 격리 테이블로

Gold · 활용

- 계정 · 일 단위 특징값 테이블
- 거래 네트워크 엣지 테이블
- 탐지 결과 · 위험 점수
- 품질 스코어카드

처리할 오염 9가지

오염	현실에서의 원인	Silver 처리	검증 방법
중복 이벤트	네트워크 재전송	event_id 기준 중복 제거	중복 건수 = 정답지
시간 역전	클라이언트 시계 오차	server_ts 기준 재정렬, 차이가 큰 행 플래그	역전 건수
타임존 혼재	서버별 로컬 시각 기록	server_id별 오프셋으로 UTC 통일	서버별 시간 분포
logout 누락	강제 종료 · 크래시	30분 비활동이면 세션 종료로 간주	세션 길이 분포
스키마 버전 혼재	패치로 필드명 변경	client_ver별 컬럼 매핑	패치 전후 건수
아이템 ID 변경	패치로 아이템 개편	매핑 테이블로 신규 ID 통일	매핑 실패 0건
골드 음수 · 오버플로	서버 버그	범위 밖 값 플래그 후 집계 제외	플래그 건수
GM · 테스트 계정	운영 계정 혼입	계정 유형으로 분석 대상에서 제외	제외 계정 수
좌표 이탈	맵 밖 좌표, 텔레포트 버그	맵 경계 검증 후 플래그	이탈 건수

3 탐지 로직

필수 · 규칙

명확한 신호

- **매크로:** 행동 간격 편차가 아주 작음, 하루 20시간 이상 접속, 같은 좌표 반복
- **작업장:** 같은 IP · 기기의 여러 계정, 비슷한 시각에 만든 계정의 같은 동선
- **RMT:** 여러 계정의 골드가 한 계정으로 집중, 시세와 동떨어진 거래

필수 · 그래프

관계로 찾기

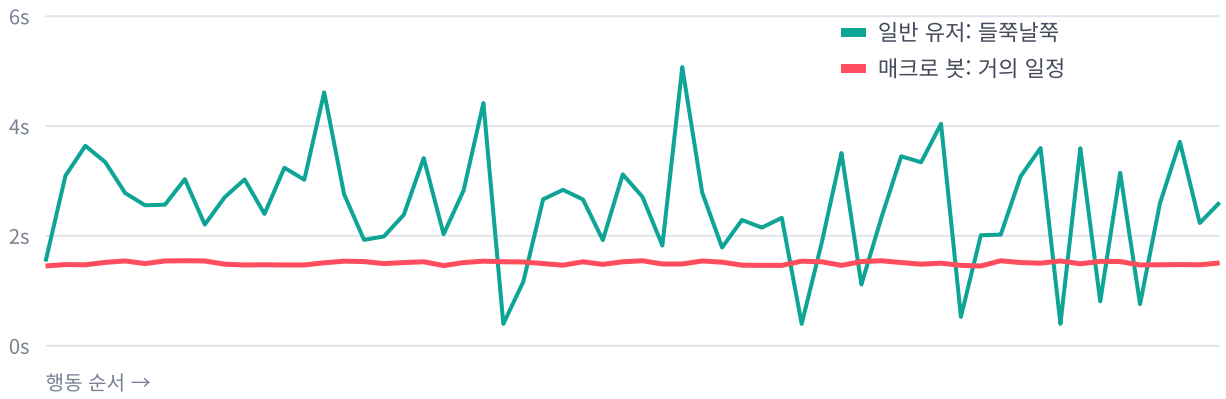
- 거래를 계정 간 네트워크로 구성
- 골드가 모이는 허브 계정 탐지
- IP · 기기로 묶이는 계정 군집

선택 · ML

시간이 남으면

- Isolation Forest로 이상치 점수
- 정답지로 규칙 기반과 성능 비교
- 헤비 유저 오탐이 줄어드는지 확인

핵심 특징값 예시: 행동 간격



예시 그림 · 사람은 행동 간격이 불규칙하지만 매크로는 거의 일정합니다. 다만 오래 플레이하는 헤비 유저는 봇과 헷갈리기 쉬워서, 실제 연구에서도 헤비 유저를 따로 나누자 오탐이 881건에서 327건으로 줄었습니다(별첨 B-4).

4 대시보드

Streamlit 한 앱에 화면 5개를 둡니다. 정제 전 · 후를 토글로 바꿔 볼 수 있게 만들어서, "정제가 탐지를 바꾼다"는 메시지를 화면에서 바로 보여 줍니다.





5 기술 스택

3일 안에 끝내야 하므로 **서버를 띄우지 않고 노트북 한 대에서 도는 구성**을 기본으로 합니다. Kafka와 Airflow는 시간이 남을 때만 붙이는 확장 항목입니다.

영역	도구	고른 이유	구분
언어 · 환경	Python 3.11, uv(또는 venv + requirements.txt)	세 명이 같은 환경을 몇 분 안에 맞출 수 있음	필수
저장 · 처리	DuckDB, Parquet	설치 없이 파일 하나로 쓰는 분석용 DB. 수천만 행도 노트북에서 SQL로 빠르게 처리	필수
데이터 가공	Polars (또는 pandas)	대용량에서 pandas보다 빠르고 메모리를 적게 씀	필수
품질 검사	SQL 규칙 파일 + pytest	규칙을 파일로 관리하고 자동 실행. 별첨 B-11의 "데이터에도 단위 테스트" 교훈을 그대로 적용	필수
네트워크 분석	NetworkX	거래 그래프의 허브 · 군집 계산	필수
대시보드	Streamlit, Plotly, pyvis	파이썬만으로 웹 대시보드를 하루 안에 만들 수 있음	필수
협업	GitHub (저장소 · Projects 칸반 · PR), Discord	작업 현황과 코드 리뷰를 한곳에서	필수
실행 자동화	Makefile (`make all`)	적재부터 대시보드까지 명령 한 줄. 재현성 확보	필수
ML	scikit-learn (Isolation Forest)	규칙 기반과 비교할 기준 모델	선택
스트리밍 시연	Kafka (Docker) + 재생 스크립트	받은 로그를 시간순으로 흘려보내 실시간처럼 시연	선택
오케스트레이션	Airflow 또는 Prefect	파이프라인을 DAG로 보여 주기	선택
배포	Docker Compose, Streamlit Community Cloud	발표 때 링크로 시연	선택

6 업무 방식

회의 리듬

매일 오전 10분 스탠드업(어제 한 일 · 오늘 할 일 · 막힌 것), 오후 늦게 15분 통합 점검.

작업 관리

GitHub Projects 칸반(할 일 → 진행 중 → 리뷰 → 완료). 카드 하나는 반나절 안에 끝나는 크기로 쪼갭니다.

Discord 음성 채널을 작업 시간 내내 열어 둡니다.

브랜치 · 리뷰

`main`은 항상 실행되는 상태로 유지. 기능별 브랜치 → PR → 다른 한 명이 확인 후 병합. 커밋 메시지는 `feat:` · `fix:` · `docs:` 형식으로 씁니다.

완료 기준

코드가 `make all`에 연결돼 있고, 품질 검사를 통과하고, README에 한 줄 이상 설명이 있으면 "완료"입니다.

데이터 계약

2차 요청서의 스키마가 곧 계약입니다. 받은 데이터가 스키마와 다르면 우리 코드를 고치기 전에 데이터 팀에 먼저 묻습니다. 질문과 답은 공유 문서에 기록합니다.

막히면 30분 규칙

30분 넘게 막히면 바로 공유합니다. 3일 프로젝트에서는 혼자 고민하는 시간이 가장 비쌉니다.

저장소 구조

```
log-watchdog/
├── data/            # 받은 원본 (Git에 올리지 않음, .gitignore)
├── pipeline/
│   ├── bronze.py    # 원본 적재
│   ├── silver.sql   # 정제 · 플래그
│   └── gold.sql     # 특징값 · 네트워크 · 집계
├── quality/
│   ├── rules.sql    # 품질 규칙
│   └── test_rules.py # pytest
├── detect/
│   ├── rules.py     # 규칙 기반 탐지
│   ├── graph.py     # 네트워크 분석
│   └── ml.py        # (선택) Isolation Forest
├── app/dashboard.py # Streamlit
├── eval/score.py    # 정답지 대비 채점
├── docs/            # 발표 자료 · 리포트
├── Makefile
└── README.md
```

7 3인 업무 분담

역할	주 담당	함께 하는 일
A · 데이터 엔지니어	환경 세팅, Bronze 적재, Silver 정제 9종, 품질 규칙과 테스트, Makefile, (선택) Kafka 재생	스키마 확인과 데이터 팀 소통 창구
B · 탐지 · 분석	Gold 특징값, 규칙 기반 탐지, 거래 네트워크 분석, 정답지 채점 스크립트, (선택) ML	정제 전후 오탐률 비교 실험
C · 시각화 · PM	Streamlit 대시보드 5개 화면, 품질 리포트, 발표 자료, README, 일정 관리와 스탠드업 진행	시연 시나리오와 리허설

데이터가 오기 전(0일차)에 C가 샘플 데이터로 대시보드 틀을 먼저 만들면, 2일차에 A · B의 결과를 바로 붙일 수 있습니다.

8 일정



날짜	목표	끝났다는 기준
0일차 (데이터 받기 전)	요청서 전달, 저장소 · 환경 세팅, 샘플 데이터로 뼈대 코드와 대시보드 틀	샘플 데이터로 `make all`이 끝까지 돈다
1일차	본 데이터 적재(Bronze), 오염 9종 정제(Silver), 품질 규칙, 특징값 설계 시작	Silver 테이블과 품질 리포트 초안이 나온다

2일차	Gold 특징값, 규칙 탐지 · 네트워크 분석, 대시보드에 실데이터 연결, 정답지 채점	정밀도 · 재현율과 정제 전후 오탐률 수치가 나온다
3일차	(선택) ML · Kafka 시연, 발표 자료 · README, 시연 영상 녹화, 리허설 2회	새 PC에서 README대로 실행된다

9 범위와 위험

필수 (MVP)

Bronze → Silver → Gold, 오염 9종 처리와 품질 리포트, 규칙 · 네트워크 탐지, 정답지 채점, 대시보드 5화면, 발표 자료.

확장 (여유 시)

Isolation Forest 비교, Kafka 실시간 재생 시연, Airflow DAG, Streamlit Cloud 배포, 실제 데이터(별첨 A)로 교차 검증.

위험	대응
데이터가 늦게 온다	0일차에 샘플(1일치)을 먼저 받아 뼈대를 완성해 둡니다. 본 데이터는 같은 스키마라 바로 갈아 끼웁니다.
데이터가 스키마와 다르다	Bronze에서 스키마 검사를 먼저 돌려 차이를 목록으로 만들고, 데이터 팀에 즉시 공유합니다. 차이 자체도 "오염"으로 리포트에 넣습니다.
데이터가 너무 크다	Parquet + DuckDB로 처리하고, 개발은 하루치로 하고 전체는 마지막에 한 번 돌립니다.
탐지 성능이 낮다	성능보다 "정제 전후 차이"와 "왜 이 규칙인가"를 설명하는 데 집중합니다. 출제자들도 점수보다 설명을 더 중요하게 봤습니다(별첨 B-11).
시연 중 오류	3일차 오후에 시연 영상을 미리 녹화해 둡니다.

10 성과 지표와 예상 질문

오탐률 변화

정제 전 대비 정제 후, 정상 유저를 봇으로 잘못 잡은 비율

정밀도 · 재현율

정답지 기준 봇 · 작업장 · RMT 탐지 성능

오염 탐지율

데이터 팀이 넣은 오염 중 우리가 찾아낸 비율

처리 시간

전체 데이터를 `make all`로 처리하는 데 걸린 시간

> **"가짜 데이터 아닌가요?"**

데이터 팀이 실제 게임사 대회(엔씨소프트 GDMC 2017, 빅콘테스트)의 로그 구조와 실제 봇 연구의 비율을 참고해 만들었습니다. 실제 데이터 출처는 별첨 A에 정리했습니다.

"게임 회사에서만 쓰는 기술 아닌가요?"

이벤트 로그 수집 · 정제 · 이상 탐지 구조는 이커머스 부정 주문, 금융 이상 거래 탐지에 그대로 적용됩니다.

"왜 처음부터 ML을 쓰지 않았나요?"

설명할 수 있는 규칙으로 기준선을 먼저 만들고, ML이 그 기준선을 얼마나 개선하는지 숫자로 비교하기 위해서입니다.